

DE LA PLUME ET DU TEXTE

Tutoriel complet de Visual Studio Code : installation, interface, extensions Python/Jupyter/débogueur, Git intégré et automatisation, en trois parcours de niveau.

- File > Open Folder : toujours ouvrir un dossier de projet, pas un fichier isolé
- Ctrl+` : ouvrir/fermer le terminal intégré, déjà positionné dans le projet
- Ctrl+Shift+X : panneau Extensions (Marketplace)
- Ctrl+Shift+P : palette de commandes, point d'entrée universel
- Ctrl+Shift+G : panneau Source Control (Git)

EXTENSION	RÔLE
Python (Microsoft)	Coloration, autocomplétion, exécution, linting/formatage (Ruff/Black)
Jupyter (Microsoft)	Notebooks .ipynb, exécution cellule par cellule, explorateur de variables
Remote-SSH	Éditer/exécuter sur un serveur distant
Dev Containers	Environnement Docker reproductible
R (REditorSupport)	Coloration/exécution R ; Quarto pour mixer Python et R

REMARQUE

PYTHON: SELECT INTERPRETER

Palette de commandes > Python: Select Interpreter — choisit l'environnement virtuel actif. Une erreur `ModuleNotFoundError` malgré un module installé vient presque toujours d'un interpréteur mal sélectionné, pas d'un module manquant.

REMARQUE

UN POINT D'ARRÊT VAUT MIEUX QU'UN PRINT()

Clic dans la marge à gauche d'une ligne pour poser un point d'arrêt, F5 pour lancer en mode débogage, F10/F11/Shift+F11 pour Step Over/Into/Out. Le panneau Variables inspecte le contenu réel à cet instant précis.

.VSCODE/TASKS.JSON : AUTOMATISER UNE CHAÎNE DE COMMANDES

```
{ "label": "Exporter", "type": "shell", "command": "python  
export.py" }
```

Déclenché par Ctrl+Shift+B (tâche par défaut) ou Tasks: Run Task.

REPRODUIRE UN ENVIRONNEMENT DE ZÉRO

```
python -m venv .venv && pip install -r requirements.txt
```

ATTENTION

À NE PAS CONFONDRE

.vscode/settings.json (paramètres d'un projet, versionnés) ≠ paramètres utilisateur globaux. .gitignore géomatique : exclure *.tif et /data/raw/, Git LFS pour les rasters de référence à versionner malgré leur poids.